Computer Organization and Assembly Language

Week 07 — Chapter 3
Comparison, Conditional Jumps and the Unconditional Jump

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 07

Outline

1. Sessional-I Solution
2. Quick Review: Before Sessional-I
3. Comparison: the CMP Instruction
4. Signed versus Unsigned
5. Flag Conditions After CMP
6. Conditional Jumps
7. Example 3.1: A Loop Without a Counter
8. The Unconditional Jump

Sessional-I Solution

Discussed separately

The complete solution key for the Sessional-I paper is handed out and walked through **in class**. It is not reproduced in these slides.

- **Q1** — physical address calculation, overlapping segments, paragraph boundaries. (CLO 2)
- **Q2** — memory capacity, organization, address and data pins. (CLO 1)
- **Q3** — memory map with mixed `db`/`dw`, byte vs word access, size mismatch. (CLO 3)
- **Q4** — program trace, `mov bx, num1` vs `mov bx, [num1]`. (CLO 3)

Bring to the discussion

Your answer sheet, and the two questions you found hardest. Marks are not the point — the point is which *concept* slipped.

Quick Review: What We Covered Before Sessional-I

- **Weeks 1–2** — computer organization vs architecture; CPU, memory and I/O joined by the **address**, **data** and **control** buses; registers, cache and the fetch–decode–execute cycle.
- **Week 3** — the `.lst` file, opcodes and instruction lengths; the **iAPX 88** register set; the **segmented memory model**,

$$\text{Physical} = (\text{Segment} \times 10\text{h}) + \text{Offset}$$

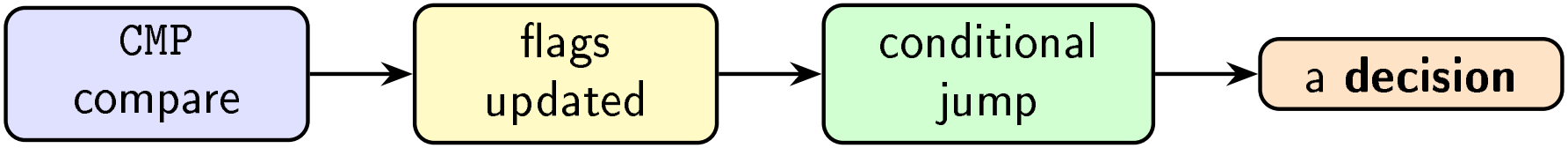
  paragraph boundaries and overlapping segments.
- **Week 4** — data declaration with `db` and `dw`, labels, and **direct addressing**.
- **Week 5** — **size mismatch** errors, **register indirect** addressing (`[bx]`), **register** + **offset** (`[num1+bx]`), the first **loop** with `sub cx,1` / `jnz`, segment association and the seven addressing modes.
- **Week 6** — Sessional-I examination.

Where Today Fits

- In Week 5 our loop ended by counting down with `CX` and testing **one** condition: *is the counter zero?*
- But real programs ask far more than that: *is this bigger? is it smaller? is it negative?*

The question of this chapter

How does a processor that can only **add** and **subtract** answer questions like *"is x greater than y?"*

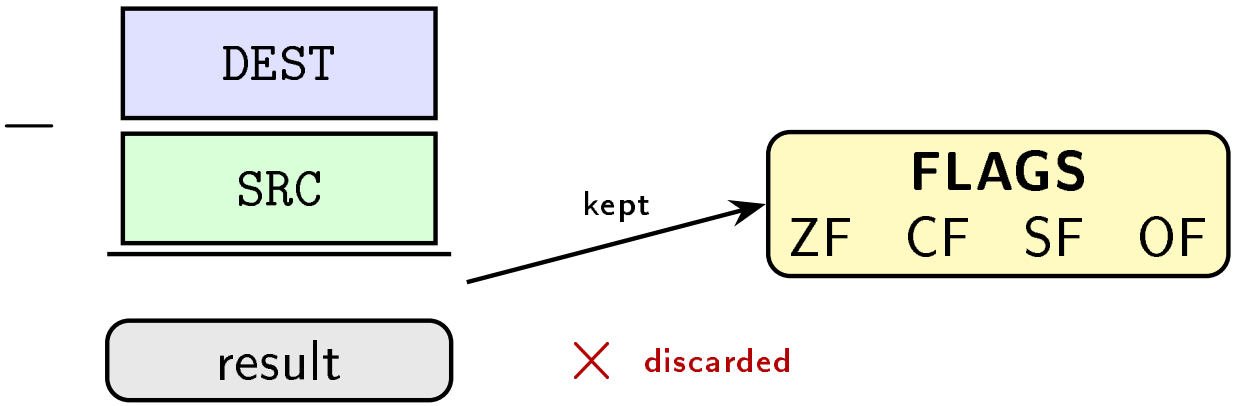


- Every `if`, `while` and `for` you have written in C++ compiles down to this chain. **They could not exist in a high-level language if they did not exist here first.**

The CMP Instruction

Definition

CMP **subtracts the source from the destination**, updates the **flags**, and throws the answer away — neither operand is changed.



- CMP is the **root instruction for all comparisons**.
- CMP is meaningless without a conditional jump immediately after it. On its own it changes nothing visible.

Why Subtraction Answers Every Question

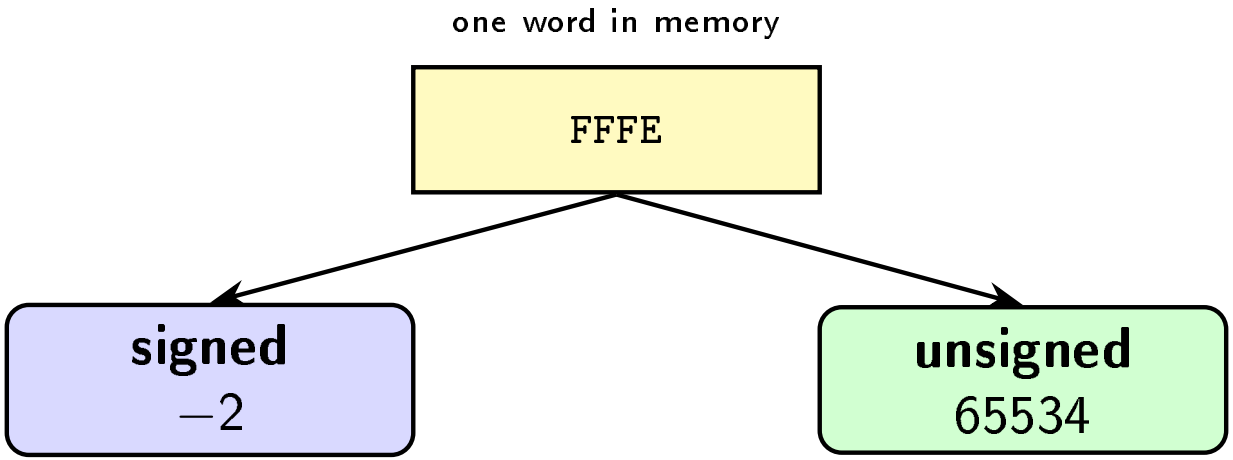
- A single subtraction reveals *every* significant relation between two operands — through the flags it leaves behind.

If ...	**then ...**	**because**
the two are equal	**ZF** $= 1$	the result is zero
DEST is smaller (unsigned)	**CF** $= 1$	a *borrow* was needed
the result is negative	**SF** $= 1$	the MSB is 1
the sign changed wrongly	**OF** $= 1$	signed overflow

- The **carry flag plays the role of borrow** during subtraction — subtracting a larger number from a smaller one requires a borrow, so CF is set.
- Four flags — **SF, CF, ZF, OF** — are enough to express *every* relation: $<$, $\leq$, $=$, $\neq$, $\geq$, $>$, for both signed and unsigned numbers.

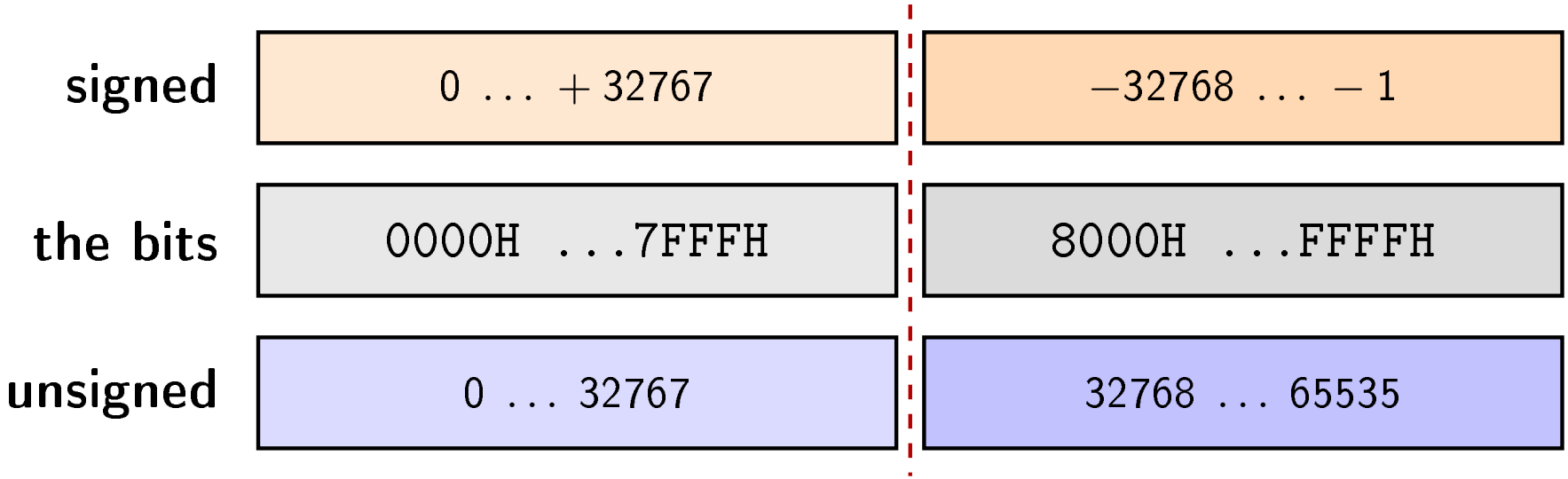
The Same Bits, Two Different Meanings

- In **unsigned** numbers only the *magnitude* matters. In **signed** numbers the *sign* matters too.
- -2 is *greater* than -3, but 2 is *smaller* than 3. The sign has reversed the comparison.



- Signed numbers are stored in **two's complement**. A number in this form is *still just a number* — only our *interpretation* is signed.
- In a byte: $2 \to 02$, $-2 \to \texttt{FE}$. Same magnitude, completely different bit pattern.

Two Orderings of the Very Same 65,536 Patterns



the whole difference is how this *right-hand half* is read

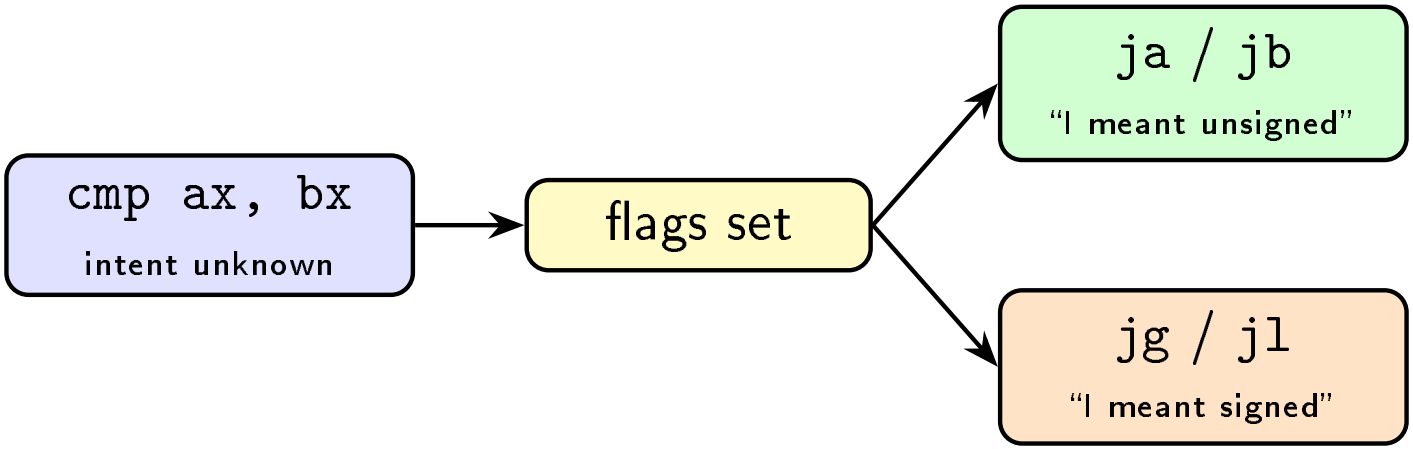
- The bit patterns are **identical**. Only the *interpretation* of the top half changes: big positive numbers, or negative numbers.
- This is why the **same** `CMP` needs two different families of jump.

When Is the Intent Declared?

The key idea

At the time of the `CMP`, the programmer's intent is **not** known. The subtraction inside `CMP` is an ordinary subtraction.

The intent is declared **later — by which conditional jump you choose**.



- `JA/JB` (above / below) read the flags the **unsigned** way.
- `JG/JL` (greater / less) read the flags the **signed** way.
- They are not interchangeable — using the wrong family silently produces wrong decisions.

Worked Example: cmp with −2 and 2

```
mov ax, -2 ; AX = FFFE
mov bx, 2 ; BX = 0002
cmp ax, bx ; FFFE - 0002
```

If we now write JA

Unsigned view:
DEST = FFFE = 65534
SRC = 0002 = 2

65534 > 2 ⇒ **jump IS taken**

If we now write JG

Signed view:
DEST = FFFE = −2
SRC = 0002 = +2

−2 < 2 ⇒ **jump is NOT taken**

- **Same comparison, same flags, opposite decisions.**
- If −2 and 65534 are compared with each other, the processor sets **ZF** — it reports them as *exactly equal*, because in memory they are the identical bit pattern.

Conditions for **Unsigned** Operands

Relation	Flags	Reason
DEST $=$ SRC	ZF $= 1$	Equal operands subtract to zero. *Works for signed and unsigned alike.*
UDEST $<$ USRC	CF $= 1$	The destination is smaller, so a **borrow** was needed — which sets the carry flag.
UDEST $\leq$ USRC	ZF $= 1$ **OR** CF $= 1$	Either they were equal (ZF), or a borrow was needed (CF).
UDEST $\geq$ USRC	CF $= 0$	No borrow is needed when the operands are equal or the destination is larger.
UDEST $>$ USRC	ZF $= 0$ **AND** CF $= 0$	Not equal (ZF $= 0$) and no borrow taken (CF $= 0$), so the destination must be larger.

- Unsigned comparisons need only **two** flags: ZF and CF.

Conditions for **Signed** Operands

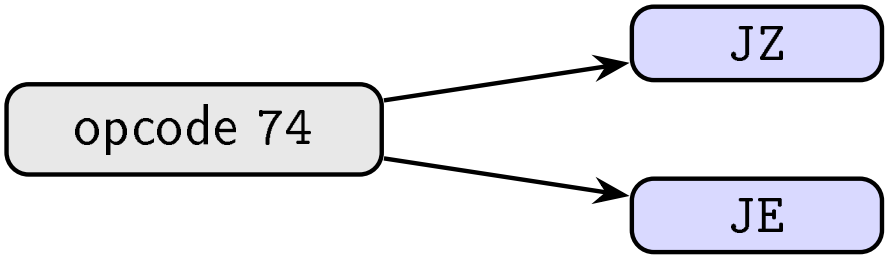
Relation	Flags	Reason
SDEST $<$ SSRC	SF $\neq$ OF	A negative answer with no overflow means the destination is smaller. If an overflow occurred, the meaning reverses and a *positive* answer signals it instead.
SDEST $\leq$ SSRC	ZF $=1$ **OR** SF $\neq$ OF	Either equal, or smaller as described above.
SDEST $\geq$ SSRC	SF $=$ OF	A positive answer with no overflow means the destination is larger; with an overflow, a negative answer signals it.
SDEST $>$ SSRC	ZF $=0$ **AND** SF $=$ OF	Not equal, and the sign and overflow flags agree.

Why SF alone is not enough

Overflow can flip the sign of the result without the true relation changing. Comparing SF *against* OF cancels that flip — so every signed condition uses the **pair**, never SF by itself.

Renamed Jumps: Many Names, One Opcode

- For every meaningful combination of flags, a conditional jump exists.
- `JZ` and `JE` are the **same instruction with the same opcode** — "jump if zero" and "jump if equal". After a `CMP`, "equal" is the natural reading; after an `ADD`, "zero" is.
- This renaming is done by **Intel** and is a standard for the iAPX 88. Some jumps have two or three names.



the debugger can only show *one* of them — it sees the opcode, not the name you typed

Jumps on the Carry and Zero Flags

Mnemonics	Meaning	Flags	Taken when
`JC JB JNAE`	carry / below / not above or equal	$CF = 1$	the last operation generated a carry or needed a borrow; after `CMP`, the unsigned destination is smaller
`JNC JNB JAE`	not carry / not below / above or equal	$CF = 0$	no carry or borrow; after `CMP`, the unsigned destination is larger or equal
`JE JZ`	equal / zero	$ZF = 1$	the last operation produced zero; after `CMP`, both operands were equal
`JNE JNZ`	not equal / not zero	$ZF = 0$	the result was non-zero; after `CMP`, the operands differ

Unsigned and Signed Jump Families

UNSIGNED — "above" and "below"		
`JA JNBE`	above / not below or equal	ZF $= 0$ **AND** CF $= 0$
`JNA JBE`	not above / below or equal	ZF $= 1$ **OR** CF $= 1$
SIGNED — "greater" and "less"		
`JL JNGE`	less / not greater or equal	SF $\neq$ OF
`JNL JGE`	not less / greater or equal	SF $=$ OF
`JG JNLE`	greater / not less or equal	ZF $= 0$ **AND** SF $=$ OF
`JNG JLE`	not greater / less or equal	ZF $= 1$ **OR** SF $\neq$ OF

A vocabulary rule worth memorising

Above / **Below** $\rightarrow$ unsigned. **Greater** / **Less** $\rightarrow$ signed.

The Remaining Jumps

Mnemonics	Meaning	Flag	Taken when
`JO` / `JNO`	overflow / not overflow	OF = 1 / 0	the last operation changed the sign unexpectedly (or did not)
`JS` / `JNS`	sign / not sign	SF = 1 / 0	the result was negative (or positive)
`JP JPE` / `JNP JPO`	parity even / odd	PF = 1 / 0	the result has an even (or odd) number of 1-bits
`JCXZ`	jump if CX is zero	**none**	`CX` = 0 — the only jump that reads no flag at all

- `JCXZ` exists because of `CX`'s special role as a **counter**. It tests the register directly, regardless of the zero flag.
- It has **no "not" counterpart** — there is no `JCXNZ`.

"Above What?" — Reading the Direction Correctly

`cmp DEST, SRC` → the flags describe the **destination relative to the source**

```
cmp ax, bx
ja  label
```

read as: **"jump if AX is above BX"**
not "if BX is above AX"

- The **first operand** is the one being described.
- `cmp ax, bx` followed by `jb` means *"jump if AX is below BX"*.
- Getting this backwards is the single most common source of off-by-one and inverted-loop bugs. **Always read the destination first.**

Example 3.1 — Adding Ten Numbers, No Counter

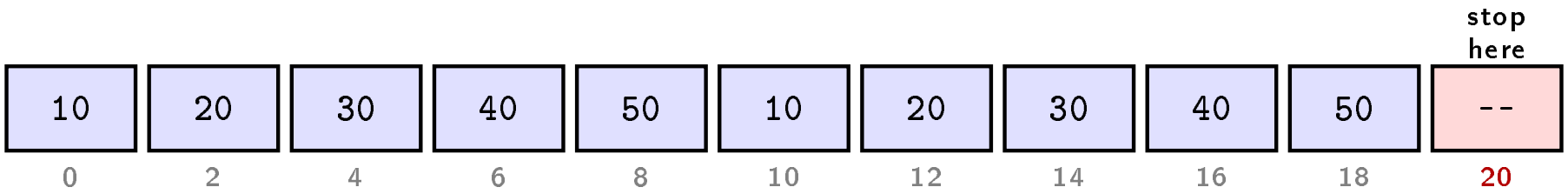
Why this example?

In Week 5 we needed **three** registers: BX for the address, CX for the count, AX for the sum. With `CMP` we can test BX directly and **free CX entirely**. It shows that a comparison can replace a whole register.

```
; a program to add ten numbers without a separate counter
[org 0x0100]
    mov bx, 0 ; initialize array index to zero
    mov ax, 0 ; initialize sum to zero
l1: add ax, [num1+bx] ; add number to ax
    add bx, 2 ; advance bx to next index
    cmp bx, 20 ; are we beyond the last index?
    jne l1 ; if not, add next number
    mov [total], ax ; write back sum in memory
    mov ax, 0x4c00
    int 0x21

num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0
```

Why the Magic Number is 20

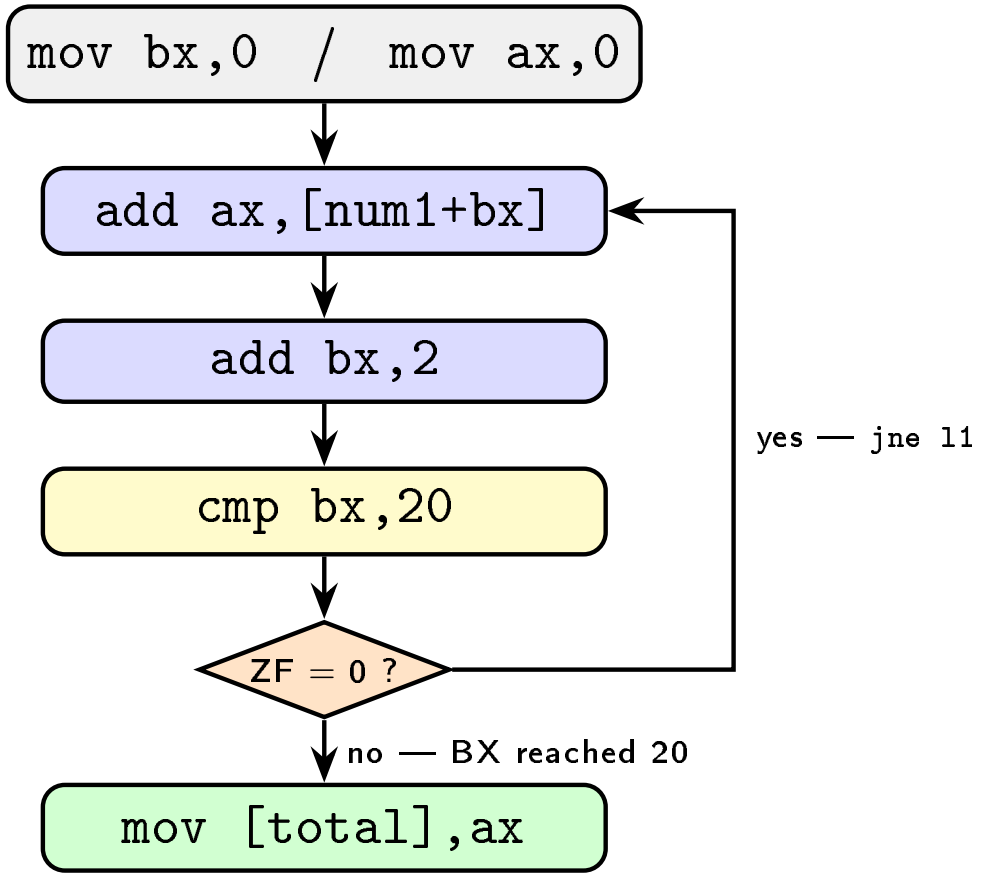


- BX is used as the **array index** *and* as the **counter** at the same time.
- The array holds 10 **words**, so the valid indices are $0, 2, 4, \ldots, 18$.
- The index of an imaginary **11th** number would be $10 \times 2 = 20$. So as soon as BX reaches 20, the 10th number has just been added and the loop stops.
- The memory access is still **base** + **offset**: `[num1+bx]` — the array base in the instruction, the index in the register.

Exam trap

If the array were declared with `db`, the stopping value would be **10**, not 20, and BX would advance by 1.

Example 3.1 — Control Flow



- `CMP` sets the flags; `JNE` reads them. Neither does anything useful alone.

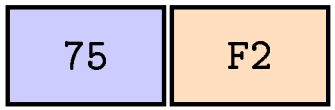
Two Things You Will Notice in the Debugger

1. Your `JNE` is displayed as `JNZ`

They are the **same opcode**. The debugger sees only the opcode byte and has no way of knowing which mnemonic you typed in the source.

2. The jump operand is `F2`, not an address

Every code and data reference so far appeared literally in the opcode. A conditional jump is different: it is a **short relative jump**. The operand is a **signed 8-bit displacement** from the *next* instruction.



`F2H` as a **signed** byte $= -14$

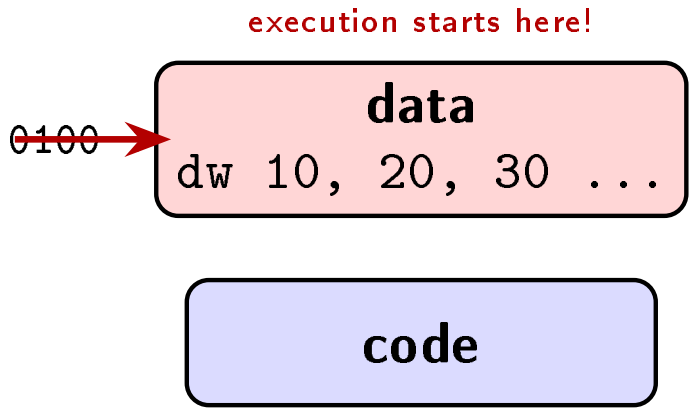
opcode `JNZ` displacement

$$\text{target} = (\text{address of the next instruction}) + (-14) \quad \rightarrow \quad \texttt{11}$$

- A negative displacement jumps **backwards** — which is exactly what a loop needs. Unconditional jumps are covered next.

A New Problem: Data at the Start of the Program

- So far we always placed data **at the end** of the code — but there is no such restriction. Data may be defined anywhere.
- Suppose we move the declarations to the **top**. Load the program in the debugger and...



- The `.COM` definition says the first executable instruction is at offset `0100`. We have put **data** there.
- The processor cannot tell data from code. It will **execute our numbers as instructions** — the debugger shows whatever meaningless opcodes it can make of them.

Example 3.2 — Jumping Over the Data

Why this example?

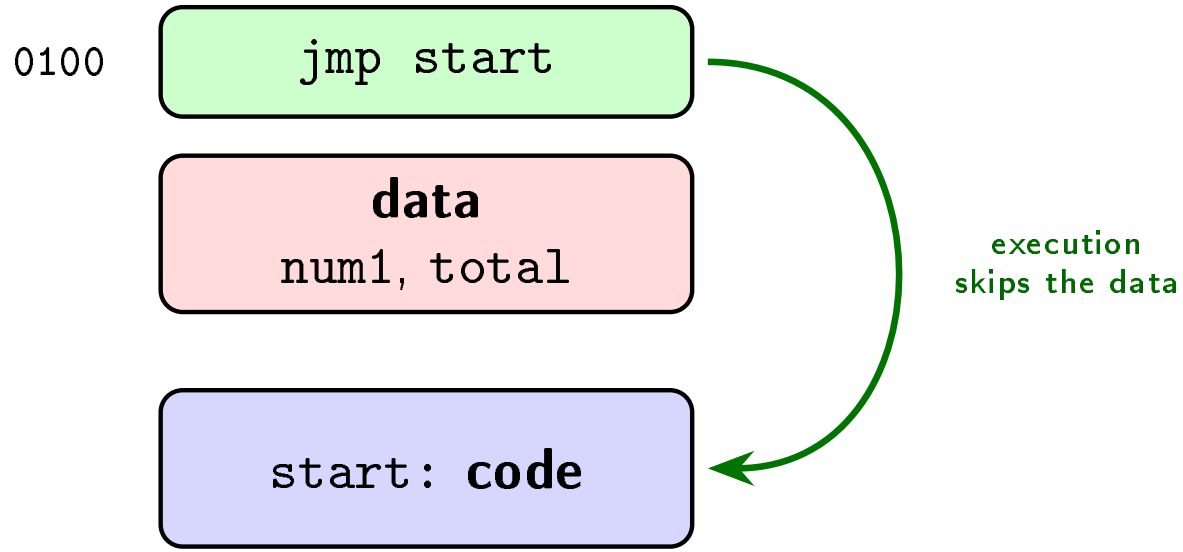
It introduces JMP, and it teaches a habit you will use in every program from now on: **when data comes first, jump over it**.

```
; a program to add ten numbers without a separate counter
[org 0x0100]
        jmp start ; unconditionally jump over data

num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0

start: mov bx, 0 ; initialize array index to zero
       mov ax, 0 ; initialize sum to zero
l1: add ax, [num1+bx] ; add number to ax
       add bx, 2 ; advance bx to next index
       cmp bx, 20 ; are we beyond the last index?
       jne l1 ; if not, add next number
       mov [total], ax ; write back sum in memory
       mov ax, 0x4c00
       int 0x21
```

How JMP Fixes It



- JMP is the **unconditional jump** — it executes regardless of the state of all flags. There is no condition to check.
- Now offset 0100 holds a valid first instruction, exactly as the .COM format requires.
- Execution resumes at start, and the data is never treated as code.

Conditional vs unconditional

A **conditional** jump chooses between two paths. An **unconditional** jump always transfers control — it is a detour, not a decision.

Summary

- `CMP` subtracts source from destination, updates the flags and **discards the result**. It is meaningless without a conditional jump after it.
- Four flags — **ZF, CF, SF, OF** — express every possible relation between two operands.
- The **same bit pattern** has two meanings: `FFFE` is -2 signed and 65534 unsigned. The `CMP` does not know which you meant.
- **The jump declares the intent:** `JA`/`JB` (above, below) $\rightarrow$ **unsigned**; `JG`/`JL` (greater, less) $\rightarrow$ **signed**.
- Unsigned conditions use **ZF and CF**; signed conditions use **ZF, SF and OF**.
- Many jumps are **renamed** by Intel — `JZ` and `JE` share one opcode, so the debugger shows only one name.
- `JCXZ` is the only jump that reads **no flag** — it tests `CX` directly.
- Conditional jumps are **short relative** jumps: the operand is a signed displacement, not an address.
- `JMP` transfers control unconditionally — use it to **jump over data** placed at the start of a program.

Thank You

Any Questions?

"A processor cannot think. It can only subtract two numbers and look at the flags.
Everything else — every decision your program ever makes — is built on top of that."

Next: unconditional jump forms, loops, and writing programs that make real decisions.